

AI Search Engine - 团队分工文档



总体概况

指标	数量
生产代码	16,075 行
测试代码	4,051 行
总代码量	~20,000 行
人均分配	~4,000 行生产代码 + ~1,000 行测试



Person 1: 基础架构 + LLM + 搜索工具



负责模块

生产代码 (3,843 行)

src/llm/	441行
├ manager.py	(LLM管理器, 多provider支持)
├ openai_client.py	(OpenAI兼容客户端)
├ ollama_client.py	(Ollama本地模型)
└ base.py	(基类)
src/utils/	1,201行
├ config.py	(配置管理, 276行)
├ logger.py	(日志系统)
├ json_logger.py	(JSON结构化日志)
├ secret_sanitizer.py	(敏感数据清理)
└ entity_extractor.py	(实体提取, 305行)
src/routing/	1,161行
├ keyword_router.py	(关键词路由, 309行)
├ llm_router.py	(LLM路由, 312行)
├ hybrid_router.py	(混合路由)
└ factory.py	(工厂类)

└─ base.py	(基类)
└─ task_types.py	(任务类型)
src/tools/ (搜索相关)	1,040行 (估算)
└─ search.py	(搜索工具)
└─ scraper.py	(网页爬取)
└─ credibility_scorer.py	(可信度评分, 320行)
└─ reranker*.py	(重排序, 3个文件)

测试代码 (~1,000 行)

tests/test_routing.py	274行
tests/test_agents.py	294行 (部分)
tests/test_performance.py	439行 (部分)

核心职责

1. LLM基础设施: 多provider管理、自动fallback、流式生成
2. 配置与日志: 全局配置加载、结构化日志、敏感信息过滤
3. 智能路由: 关键词路由、LLM路由、混合路由策略
4. 搜索工具: SerpAPI集成、网页爬取、可信度评分

接口定义

- **LLMManager**: 统一LLM接口，其他人直接调用
- **Config**: 配置读取接口
- **Router**: 路由决策接口

Person 2: 代码执行 + 安全系统

负责模块

生产代码 (4,075 行)

src/tools/ (代码执行相关)	1,307行
└─ code_executor.py	359行 (代码执行协调器)
└─ code_validator.py	423行 (AST安全验证)
└─ sandbox_executor.py	529行 (Docker沙箱)

src/agents/	179行
└─ code_agent.py	179行 (代码生成Agent)
src/tools/ (领域工具)	705行
└─ weather_tool.py	(天气API)
└─ finance_tool.py	278行 (股票数据)
└─ routing_tool.py	418行 (路线规划)
src/workflow/	1,686行
└─ workflow_engine.py	594行 (工作流引擎)
└─ task_decomposer.py	579行 (任务分解)
└─ result_aggregator.py	480行 (结果聚合)
src/main.py	302行 (CLI入口)

测试代码 (~1,200 行)

tests/test_code_security.py	563行 (安全测试)
tests/test_workflow.py	535行 (工作流测试)
tests/test_tools.py	431行 (部分工具测试)

 核心职责

- 1. 安全代码执行: 3层防护 (AST + Docker + Timeout)
- 2. **CodeAgent**: 代码生成与执行协调
- 3. 领域工具: 天气、金融、路线规划API集成
- 4. 工作流引擎: 多步骤任务编排、DAG执行

 接口定义

- **CodeExecutor**: 安全代码执行接口
- **WorkflowEngine**: 工作流执行接口
- **Domain Tools**: 天气/金融/路线查询接口

 Person 3: RAG系统 + 文档处理

 负责模块

生产代码 (4,190 行)

src/agents/	391行
└─ rag_agent.py	391行 (RAG Agent核心)
src/tools/ (RAG相关)	1,855行
└─ vector_store.py	348行 (ChromaDB向量存储)
└─ chunking.py	258行 (文档分块策略)
└─ document_processor.py	(文档处理)
└─ advanced_pdf_processor.py	573行 (智能PDF处理)
└─ ocr_tool.py	279行 (OCR文字识别)
└─ vision_tool.py	377行 (视觉理解)
src/agents/	1,292行
└─ research_agent.py	218行 (研究Agent)
└─ chat_agent.py	52行 (聊天Agent)
└─ master_agent.py	952行 (主控Agent)
src/web/	652行
└─ database.py	579行 (SQLite数据库)
└─ upload_manager.py	342行 (文件上传管理)

测试代码 (~900 行)

tests/test_complete_system.py	752行 (系统集成测试)
tests/test_agents.py	294行 (部分Agent测试)

核心职责

1. **RAG系统**: 文档索引、语义搜索、混合重排序
2. **文档处理**: PDF多模态处理、OCR、分块策略
3. **Agent协调**: ResearchAgent、ChatAgent、MasterAgent
4. **数据持久化**: SQLite历史记录、文件上传管理

接口定义

- **RAGAgent**: 文档问答接口
- **VectorStore**: 向量存储CRUD接口
- **MasterAgent**: 智能任务编排接口

Person 4: Web应用 + UI + 多模态集成

负责模块

生产代码 (3,967 行)

src/web/routers/	2,787行 (估算)
├─ query.py	(统一查询)
├─ search.py	(搜索)
├─ code.py	(代码执行)
├─ chat.py	(聊天)
├─ rag.py	352行 (RAG路由)
├─ multimodal.py	372行 (多模态路由)
├─ tools.py	451行 (工具路由)
├─ workflow.py	372行 (工作流路由)
├─ history.py	(历史记录)
└─ main.py	(主页)
src/web/dependencies/	521行
├─ core.py	270行 (核心依赖)
├─ tools.py	251行 (工具依赖注入)
└─ formatters.py	(格式化)
src/web/middleware/	~200行
├─ rate_limiter.py	(限流中间件)
src/web/	171行
├─ app.py	(FastAPI应用入口)
└─ templates/	(HTML模板)
src/web/static/	(CSS/JS静态资源)

测试代码 (~950 行)

tests/test_web_api.py	81行
tests/test_load.py	403行 (负载测试)
tests/test_performance.py	439行 (性能测试)
tests/archive/test_web_ui.py	125行 (UI测试)

核心职责

1. **FastAPI应用**: REST API、SSE流式响应、CORS、限流
2. **路由层**: 10+ API端点实现
3. **依赖注入**: 工具实例管理、生命周期
4. **前端UI**: HTML模板、静态资源、交互逻辑

接口定义

- **REST API endpoints** (所有对外接口)
 - **WebSocket/SSE streaming**
 - 文件上传接口
-

17 开发时间线建议

Phase 1: 基础设施 (Week 1-2)

- **Person 1:** LLM Manager + Config + Logger 
- **Person 2:** Code Validator + Sandbox (基础)
- **Person 3:** VectorStore + Chunking (基础)
- **Person 4:** FastAPI App 骨架 + 中间件

里程碑: LLM可用、配置加载、基础安全

Phase 2: 核心功能 (Week 3-4)

- **Person 1:** Routing System + Search Tools
- **Person 2:** Code Executor + CodeAgent 完整实现
- **Person 3:** RAG Agent + Document Processor
- **Person 4:** API Routers (query, search, code, chat)

里程碑: 查询路由、代码执行、文档问答

Phase 3: 高级特性 (Week 5-6)

- **Person 1:** Credibility Scorer + Reranker
- **Person 2:** Workflow Engine + Domain Tools
- **Person 3:** Advanced PDF + OCR + Vision
- **Person 4:** Multimodal/RAG/Tools/Workflow Routers

里程碑: 工作流、多模态、领域工具

Phase 4: 集成测试 (Week 7-8)

- **All:** 编写单元测试 + 集成测试
- **Person 1:** test_routing.py
- **Person 2:** test_code_security.py + test_workflow.py
- **Person 3:** test_complete_system.py
- **Person 4:** test_web_api.py + test_load.py

里程碑: 80%+ 测试覆盖率

协作接口规范

Person 1 提供给所有人:

```
# LLM接口
llm_manager = get_llm_manager()
response = await llm_manager.generate(messages, temperature=0.7)

# 配置接口
config = get_config()

# 路由接口
router = create_router(config, llm_manager)
decision = await router.route(query)
```

Person 2 提供给 Person 3, 4:

```
# 代码执行
executor = get_code_executor()
result = await executor.execute(code, language="python")

# 工作流
engine = WorkflowEngine()
result = await engine.execute_workflow(tasks)
```

Person 3 提供给 Person 4:

```
# RAG查询
rag_agent = get_rag_agent()
answer = await rag_agent.query(question)

# 文档上传
await rag_agent.ingest_documents(file_path)
```

Person 4 提供给所有人:

```
# API测试
response = client.post("/query", json={"query": "test"})
```

关键注意事项

并行开发要点:

1. **Person 1** 优先启动 (基础设施阻塞其他人)
 - Week 1 完成 LLM Manager + Config
 - 其他人使用 Mock 开始开发
2. 定义清晰接口
 - 每个模块提供 `__init__.py` 导出
 - 使用类型注解 (虽然目前缺失)
 - 接口先行, 实现后补
3. 避免循环依赖
 - Utils/LLM → Tools → Agents → Web (单向依赖)
 - 不允许 Web 层直接调用 Tools (通过 Agent)
4. 共享测试策略
 - Person 1: Mock LLM responses
 - Person 2: Docker测试环境
 - Person 3: 样本文档库
 - Person 4: API契约测试



工作量验证

Person	生产代码	测试代码	总计	核心模块数
Person 1	3,843	1,000	4,843	LLM + Utils + Routing + Search
Person 2	4,075	1,200	5,275	Code + Security + Workflow
Person 3	4,190	900	5,090	RAG + Docs + Agents
Person 4	3,967	950	4,917	Web + UI + Integration
总计	16,075	4,050	20,125	-

工作量偏差: ±5% (非常均衡)

最终建议

1. 技能匹配:

- **Person 1:** Python基础设施、设计模式、配置管理
- **Person 2:** 系统安全、Docker、并发编程
- **Person 3:** 机器学习、NLP、向量数据库
- **Person 4:** Web开发、FastAPI、前端基础

2. 代码审查:

- Person 1 ↔ Person 3 (相互review Agent相关)
- Person 2 ↔ Person 4 (相互review 安全和API)